

Intro to AI Engineering

Satya Pattnaik

→ Modern Text/Image models

GPT2 → GPT4.5/Llama/Deepseek

ViT → Vision Transformers (Image)

→ Proprietary LLM APIs (GPT, OpenAI)

→ Eval Pipelines ()

→ Prompt Engineering

→ RAG (deep dive)

→ Embeddings (Finetune/train one Embedding Models)
 Finetune one RAG system
 Bi-Encoder
 Cross-Encoder
 BERT

→ Multimodal/Tabular RAG

→ Agents (Multiple classes)
 Today Guide intro

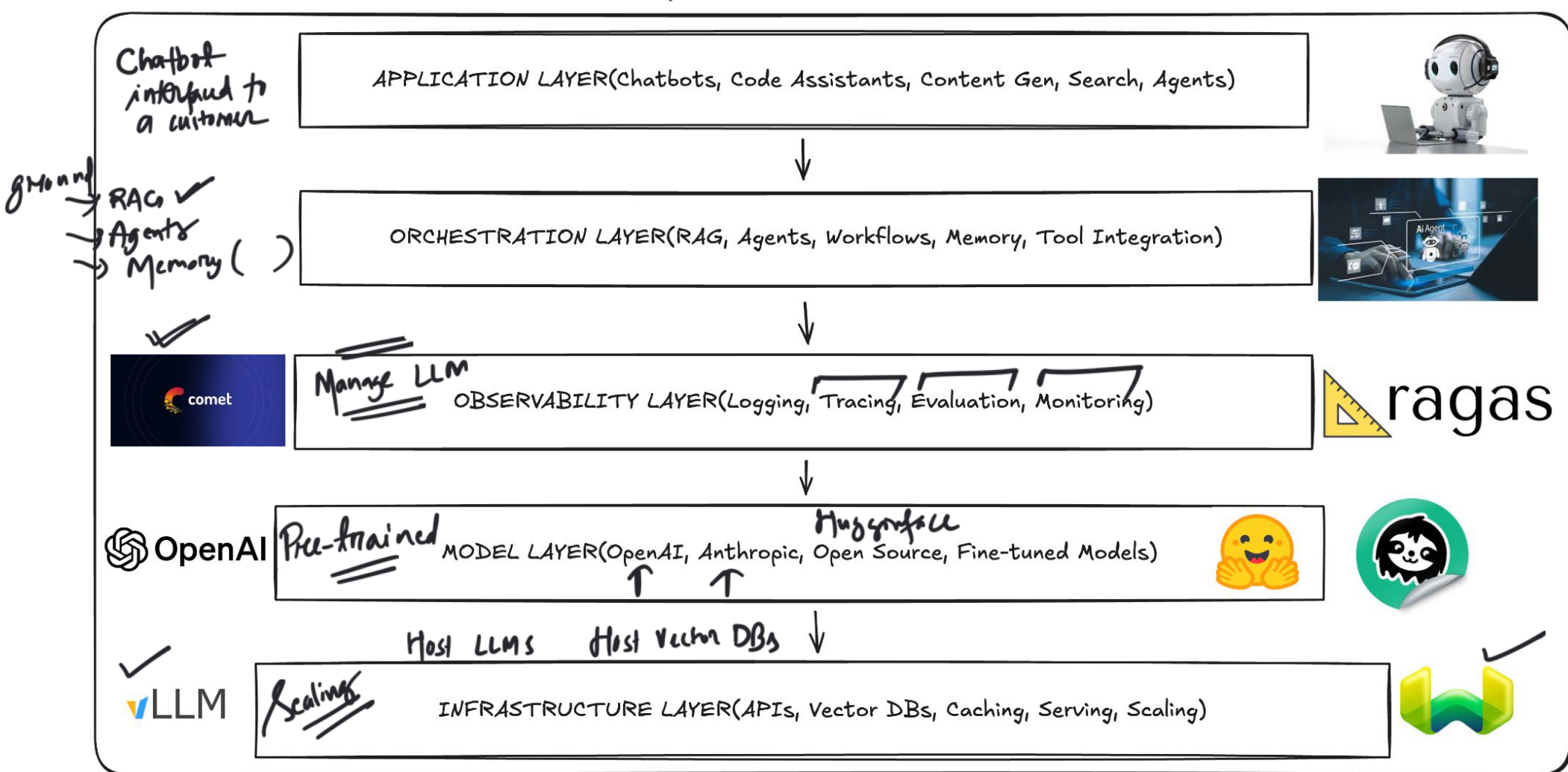
→ Scale the AI applications
 Scaling vector DB
 Scaling the serving of LLM

→ Quantization

→ Finetuning Model
 Simple
 Advanced - Train a LLM on multi-node SGPUS

→ Inference optimization - vLLMs

Layers of a AI Engg Stack



ML Engineering vs AI Engineering

ML Engineering

Primary Focus → Training models from scratch

Model Development → Architecture design, hyperparameter tuning

Data Requirements → Large labeled datasets required

Iteration Cycle → Days to weeks per experiment

Key Skills → Statistics, ML algorithms, distributed training

Compute Costs → Training compute dominant

Evaluation → Accuracy, F1, AUC on held-out sets

Production → Model versioning, retraining pipelines

AI Engineering

Primary Focus → Building apps with foundation models

Model Development → Prompt engineering, fine-tuning, RAG

Data Requirements → Examples for prompts, retrieval corpus

Iteration Cycle → Minutes to hours per iteration

Key Skills → Prompt design, system architecture, evals

Compute Costs → Inference costs dominant

Evaluation → LLM-as-judge, human eval, task-specific

Production → Latency, cost optimization, safety

Actual LLM needed to build your LLM app

proprietary



proprietary



open source

→ Customize
→ host them according to your specification



Hugging Face

OPUS 4.5

no GPUs

POC - OpenAI model

Scale your POC - Open source ← maintain
GPU requirements

Prompts - Interface betⁿ LLMs/Human

RAG

Vector DB

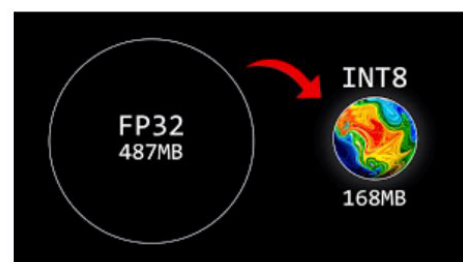


Chroma



Weaviate

Quantization



Evaluation → Multiple LLMs (benchmarking)



ragas



opik

LLM
Eval
tools

tracked

Experiment Tracking



comet



EVIDENTLY AI

GROUNDING A LLM

Context

Why is a RAG needed?

LLMs - OpenAI
trained on open source data

Will LLM have this info? NO

~~FAQS~~
Internal docs

25th

proprietary

provide as a context -> 25th

X - When will I get my salary?

LLM -> 30th of every month

Retrieval
(internal)
docs

Augmented Generation

RAG

→ 1 million pages of internal Docs/FAQs

millions/billions of tokens

LLM can't read all

30K words → LLM

✓ RAG

9:10

How to implement RAG?

→ LLM

→ Vector Database
→ Embedding Model

→ Text Splitting -

Big doc → Break them into sub docs

* Vector DBs are refreshed / updated using CRON jobs

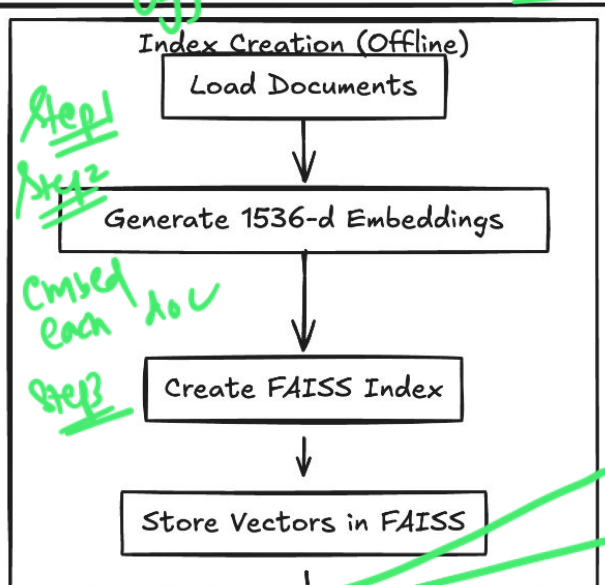
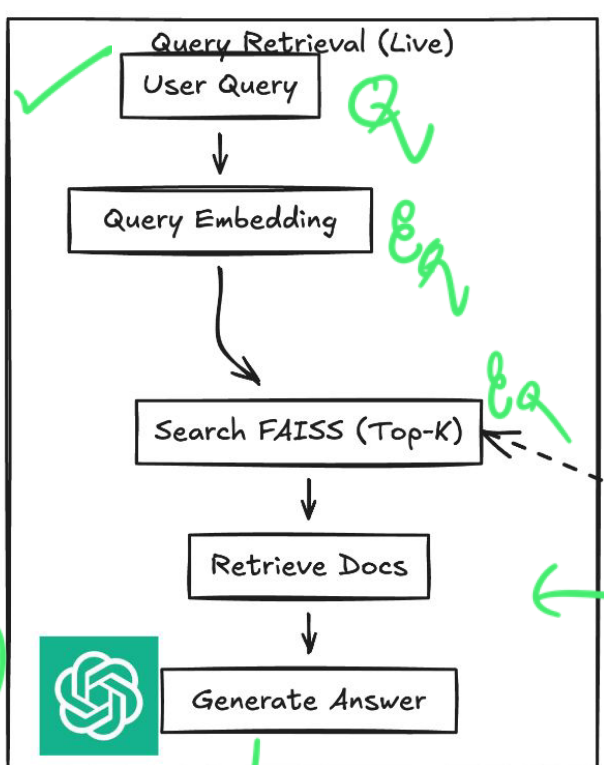
* RAG doesn't have info about query ⇒ hallucinates
* How good of a grounding system you can build

Need of RAG

- LLM doesn't know your private data
- LLMs also can't learn infinite info
- Ground LLMs (they don't hallucinate)

1536-d } depends on embedding model
↑
can change

Offline Precreated



Vector DB Setup

E_{DOC1}
 E_{DOC2} 0.77 DOC2
:
 E_{DOCN}

Shared FAISS Index

Q + Context (VDB)



✓
Answer

Now LLM is grounded

Retrieve doc and pass to LLM

* 1536-d is only specific to our model
it can change based on model BERT x

* OpenAI LLM $\longleftrightarrow$ Embedding
need not
be if
same provider
Vector DB and your Query embedding
model should be same

Vector DB - BERT
Query - BERT
BERT x
OpenAI x

Q (Query)

Cosine - similarity [vector info]

Embedding(Doc1) — Vector1

Doc2 — Vector2

⋮

Doc N → Vector N

Vector DB

Key	Value
Vector1(Doc1)	Doc1
Vector2(Doc2)	<u>Doc2 + Q</u> Answer
⋮	
Vector N(DocN)	DocN

are not
passed to LM

1 million request per day

Context (RAG)

+ User Q

→ LLM_G → response

↑
smaller
cheaper
open source

GPT4-o-mini
Llama 3.1

Faithfulness

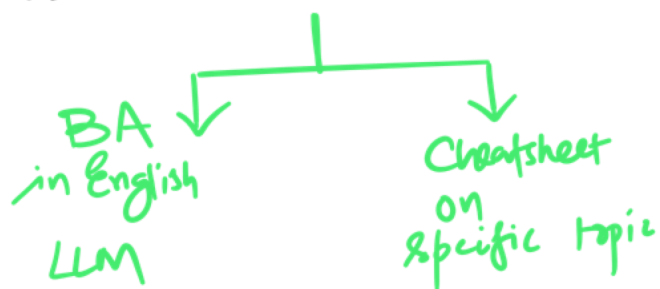
How good your LLM_G response
is grounded with
the context?

Validation set → 100



expensive { LLM_{judge}
powerful (GPT 5 Reasoner)
(Anthropic Opus 4.5)

Do we use a RAG or



we finetune?
 <train>

Answer - Evaluation

↓ BA in English
↓ 2 years
specializing then
on a field

* Q LoRA
* LoRA } finetune
 } cheaper
 ↻ constant

Benefits
→ enables customization ✓

* Disclaimer

RAG and finetune need not be complementary

[RAG + Finetune] ✓

Cons

training efforts

high quality training data

GPU

AG ops

Privacy / Safety

Gen AS app

→ Retrieval

→ Memory (user chat)

→ Hook

→ structured of

→ long conv

Orchestrated - Langchain

$P(\text{query})$



$P(\text{doc})$

Context



doc

+

User
 Q



→ LLM

→

Response

Orchestration



Agent



LLM Finetuning



Prompt Optimization



$$\underline{5 * 3} + 2$$

Reasoning - BODMAS

✓ — Multiply

✓ — AAA

$$\checkmark \{ 55555 * 3333 + 2222 \}$$

Reasoning

→ Multiply

→ AAA

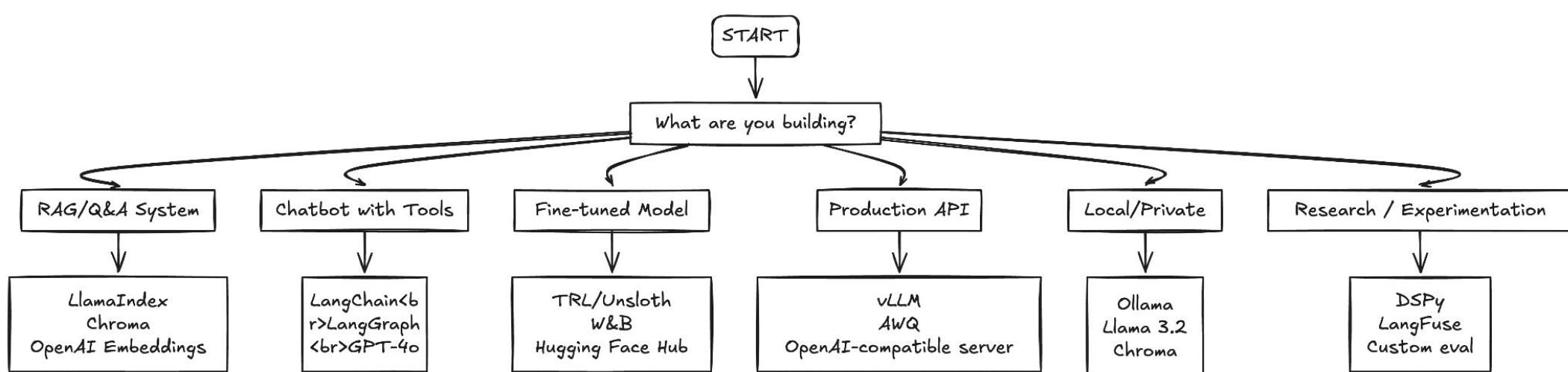
reason

$$\{ \text{Calc}(55555 * 3333) \}$$

$$\{ \text{Calc}(9999 + 2222) \}$$

LLM (Brain)
Tools (Limbs)

1] Reasoning/Planning
2] Calculation/Tool Call } Agent



Kusto SQL ✓✓
↑

Llama — [✓]RL → LLM

Nested joins — ()

~~GPT2~~

{ Agents — OpenAI GPT4.5 ✓✓
Llama 70B ✓✓

ML workflow



Agents

Decision — LLM ✓ (tools)

Workflow is not human dependant

Quantizatⁿ

Prompt

English

Context

How good of context

RAG

+
Agentic Workflow

MCP - Standardized way of providing access to
tools for Agents